

Aufgabe 1

Eine automatisierte Stanzmaschine, siehe Abbildung 1, besteht aus zwei Hauptkomponenten:

- **Förderbandsystem:**

Das Förderband transportiert Werkstücke durch die Anlage. Es wird von zwei separaten Motoren angetrieben: Ein $x_FoerderbandVorwaerts$ bewegt das Band in die Stanzrichtung, während ein $x_FoerderbandRueckwaerts$ das Band in die entgegengesetzte Richtung bewegt. Zur Werkstückerkennung sind zwei Lichtschranken positioniert. Eine Lichtschranke $x_EinAuslagerung$ befindet sich am Anfang des Förderbands, und eine Lichtschranke $x_Stanzposition$ ist dem Stanzmechanismus vorgelagert, wodurch die finale zeitgesteuerte Positionierung des Werkstücks für den Stanzvorgang eingeleitet werden kann.

- **Stanzmechanismus:**

Dieser Mechanismus führt den eigentlichen Bearbeitungsvorgang am Werkstück durch. Er wird von zwei Motoren angetrieben. Ein Motor $x_StanzeHoch$ hebt den Stanzstempel, und ein Motor $x_StanzeRunter$ senkt ihn zum Stanzen. Die Position des Stanzstempels wird durch zwei Endlagen-Sensoren überwacht. Eine Endlage $x_StanzeOben$ signalisiert, wenn der Stempel vollständig angehoben ist, und eine Endlage $x_StanzeUnten$ zeigt an, wenn der Stempel die unterste Position erreicht hat.

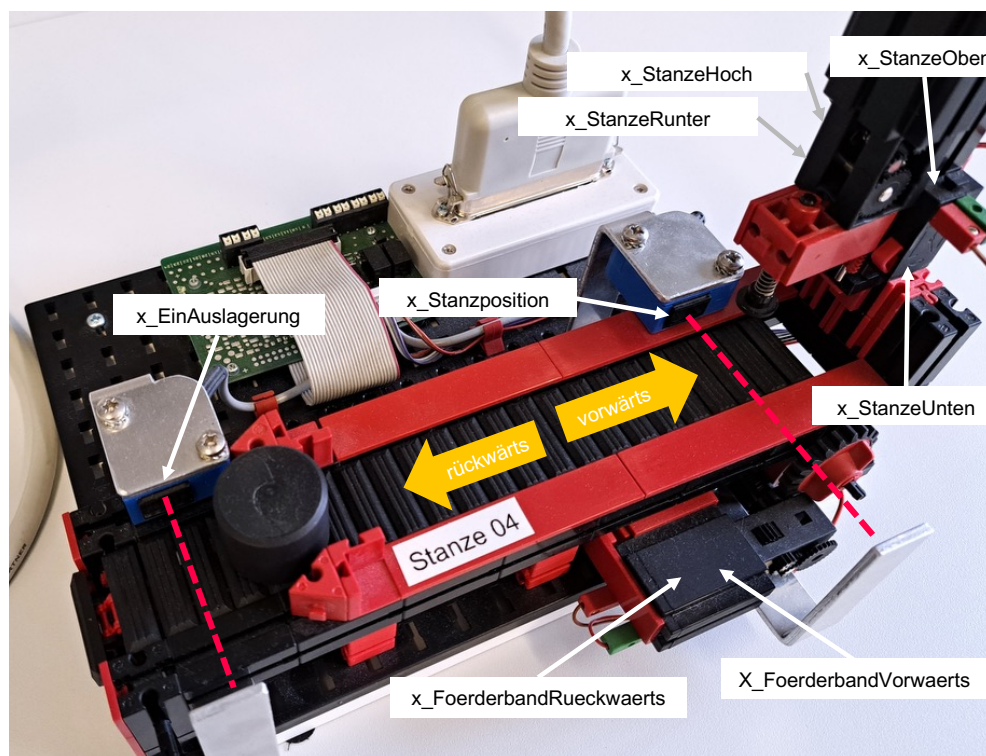


Abbildung 1: Anordnung der Sensoren und Aktoren in einer Stanzmaschine

Hinweis zur Sensorlogik:

Die beiden Lichtschranken (*x_EinAuslagerung* und *x_Stanzposition*) sind als Öffner (Normally Closed, NC) ausgeführt. Dies bedeutet, dass der Ausgang den Wert FALSE (0) liefert, wenn der Lichtstrahl durch ein Werkstück unterbrochen wird. Im Gegensatz dazu liefert der Sensor den Wert TRUE (1), wenn kein Werkstück vorhanden ist und der Lichtstrahl frei durchgehen kann. Diese inverse Logik muss bei der Programmierung berücksichtigt werden.

Aufgabe 1.1

Programmcode analysieren (10 Minuten)

In dieser Aufgabe lernen Sie die Grundfunktion der Stanzmaschine kennen, indem Sie einen bereits implementierten und funktionsfähigen Funktionsbaustein analysieren. Der vorliegende Code ist fehlerfrei übersetzbar und kann direkt in PLCSim ausgeführt werden. Ihre Aufgabe besteht darin, die Funktionsweise des Bausteins zu verstehen und Fragen zum Code zu beantworten.

Fragen zum Code

Bitte beantworten Sie die folgenden Fragen, um Ihr Verständnis des Funktionsbausteins FB11 und des beinhalteten Programm-Codes zu überprüfen. Die Fragen beziehen sich auf spezifische Zeilen und Kommentare im Code.

Lesen Sie jede Frage sorgfältig durch. Für jede Frage ist nur eine Antwort korrekt.

1) In den Zeilen 26 und 27 wird eine Flankenerkennung für den Eingang *x_Ausfuehren* implementiert. Welchen Zweck erfüllt dieser Mechanismus hier?

- [A] Er stellt sicher, dass der Prozess nur startet, wenn *x_Ausfuehren* dauerhaft auf TRUE gehalten wird.
- [B] Er verhindert einen Neustart des Prozesses, solange die Variable *x_Prozess_aktiv* noch TRUE ist.
- ☒ [C] Er wandelt das statische *x_Ausfuehren*-Signal in einen einmaligen Impuls (*x_Flanke*) um, damit der Prozess bei jedem neuen Betätigen exakt einmal angestoßen wird.
- [D] Er zählt, wie oft der *x_Ausfuehren*-Eingang betätigt wurde, um die Anzahl der Stanzvorgänge zu protokollieren

/2

2) Betrachten Sie den IF-Block von Zeile 29 bis 35. Wenn der Prozess durch eine positive Flanke an *x_Ausfuehren* gestartet wird, welche der folgenden Variablen wird als Einzige auf TRUE gesetzt, während die anderen explizit auf FALSE gesetzt werden?

[A] *x_Fertig*

☒ [B] *x_Prozess_aktiv*

[C] *x_Abwaerts_fertig*

[D] *x_Haltzeit_aktiv*

/2

3) Der Code in den Zeilen 39 bis 48 steuert die Abwärtsbewegung. Laut dieser Logik, was ist die direkte auslösende Bedingung, die erfüllt sein muss, damit die Variable *x_Haltzeit_aktiv* auf TRUE gesetzt wird?

[A] Die Variable *x_Abwaerts_fertig* muss den Zustand TRUE annehmen.

[B] Der Ausgang *x_Stanze_runter* muss auf FALSE gesetzt werden.

☒ [C] Der Sensor *x_Stanze_unten* muss ein TRUE-Signal liefern.

[D] Die übergeordnete Variable *x_Prozess_aktiv* muss TRUE sein.

/2

4) Die Aufwärtsbewegung wird in den Zeilen 57 bis 65 gesteuert. Welche Aussage beschreibt die Logik zum Beenden der Aufwärtsbewegung (*x_Stanze_hoch*) korrekt und präzise gemäß dem Code?

[A] Die Aufwärtsbewegung stoppt, sobald der Timer *time_TON_Haltzeit* seinen Zählvorgang beendet hat.

☒ [B] Die Aufwärtsbewegung stoppt, indem *x_Stanze_hoch* auf FALSE gesetzt wird, sobald der obere Endlagenschalter (*x_Stanze_oben*) TRUE wird.

[C] Die Aufwärtsbewegung stoppt, wenn die Statusvariable *x_Aufwaerts_fertig* den Wert TRUE annimmt.

[D] Die Aufwärtsbewegung stoppt, weil die Bedingung *NOT x_Haltzeit_aktiv* in Zeile 58 nicht mehr erfüllt ist.

/2

5) Betrachten Sie den isolierten Codeabschnitt in den Zeilen 77 bis 80. Welche spezifische Funktionalität wird durch diesen Code-Teil realisiert, der außerhalb der normalen Schrittkette des Prozesses steht?

- [A] Eine manuelle Reset-Funktion, um den Stempel bei einem Fehler in die obere Grundposition zu zwingen.
- ☒ [B] Eine Sicherheitsfunktion, die widersprüchliche Steuersignale auflöst, indem sie bei gleichzeitigem Ansteuern beider Richtungen, die Bewegung stoppt.
- [C] Die Initialisierung des Stempels, die sicherstellt, dass `x_Stanze_hoch` zu Beginn FALSE ist.
- [D] Ein Testmodus, der prüft, ob die Motoren für `x_Stanze_hoch` und `x_Stanze_runter` korrekt angeschlossen sind.

✓
2